

496

当定义一个类时，我们显式地或隐式地指定在此类型的对象拷贝、移动、赋值和销毁时做什么。一个类通过定义五种特殊的成员函数来控制这些操作，包括：拷贝构造函数（copy constructor）、拷贝赋值运算符（copy-assignment operator）、移动构造函数（move constructor）、移动赋值运算符（move-assignment operator）和析构函数（destructor）。拷贝和移动构造函数定义了当用同类型的另一个对象初始化本对象时做什么。拷贝和移动赋值运算符定义了将一个对象赋予同类型的另一个对象时做什么。析构函数定义了当此类型对象销毁时做什么。我们称这些操作为拷贝控制操作（copy control）。



如果一个类没有定义所有这些拷贝控制成员，编译器会自动为它定义缺失的操作。因此，很多类会忽略这些拷贝控制操作（参见 7.1.5 节，第 239 页）。但是，对一些类来说，依赖这些操作的默认定义会导致灾难。通常，实现拷贝控制操作最困难的地方是首先认识到什么时候需要定义这些操作。



在定义任何 C++ 类时，拷贝控制操作都是必要部分。对初学 C++ 的程序员来说，必须定义对象拷贝、移动、赋值或销毁时做什么，这常常令他们感到困惑。这种困扰很复杂，因为如果我们不显式定义这些操作，编译器也会为我们定义，但编译器定义的行为可能并非我们所想。

13.1 拷贝、赋值与销毁

我们将以最基本的操作——拷贝构造函数、拷贝赋值运算符和析构函数作为开始。我们在 13.6 节（第 470 页）中将介绍移动操作（新标准所引入的操作）。



13.1.1 拷贝构造函数

如果一个构造函数的第一个参数是自身类类型的引用，且任何额外参数都有默认值，则此构造函数是拷贝构造函数。

```
class Foo {
public:
    Foo();           // 默认构造函数
    Foo(const Foo&); // 拷贝构造函数
    // ...
};
```

拷贝构造函数的第一个参数必须是一个引用类型，原因我们稍后解释。虽然我们可以定义一个接受非 const 引用的拷贝构造函数，但此参数几乎总是一个 const 的引用。拷贝构造函数在几种情况下都会被隐式地使用。因此，拷贝构造函数通常不应该是 explicit 的（参见 7.5.4 节，第 265 页）。

497

合成拷贝构造函数

如果我们没有为一个类定义拷贝构造函数，编译器会为我们定义一个。与合成默认构造函数（参见 7.1.4 节，第 235 页）不同，即使我们定义了其他构造函数，编译器也会为我们合成一个拷贝构造函数。

如我们将在 13.1.6 节（第 450 页）中所见，对某些类来说，合成拷贝构造函数（synthesized copy constructor）用来阻止我们拷贝该类类型的对象。而一般情况，合成的拷贝构造函数会将其参数的成员逐个拷贝到正在创建的对象中（参见 7.1.5 节，第 239 页）。编译器从给

定对象中依次将每个非 static 成员拷贝到正在创建的对象中。

每个成员的类型决定了它如何拷贝：对类类型的成员，会使用其拷贝构造函数来拷贝；内置类型的成员则直接拷贝。虽然我们不能直接拷贝一个数组（参见 3.5.1 节，第 102 页），但合成拷贝构造函数会逐元素地拷贝一个数组类型的成员。如果数组元素是类类型，则使用元素的拷贝构造函数来进行拷贝。

作为一个例子，我们的 Sales_data 类的合成拷贝构造函数等价于：

```
class Sales_data {
public:
    // 其他成员和构造函数的定义，如前
    // 与合成的拷贝构造函数等价的拷贝构造函数的声明
    Sales_data(const Sales_data&);
private:
    std::string bookNo;
    int units_sold = 0;
    double revenue = 0.0;
};
// 与 Sales_data 的合成的拷贝构造函数等价
Sales_data::Sales_data(const Sales_data &orig):
    bookNo(orig.bookNo),           // 使用 string 的拷贝构造函数
    units_sold(orig.units_sold),   // 拷贝 orig.units_sold
    revenue(orig.revenue)          // 拷贝 orig.revenue
{ }                                // 空函数体
```

拷贝初始化

现在，我们可以完全理解直接初始化和拷贝初始化之间的差异了（参见 3.2.1 节，第 76 页）：

```
string dots(10, '.');           // 直接初始化
string s(dots);                 // 直接初始化
string s2 = dots;               // 拷贝初始化
string null_book = "9-999-99999-9"; // 拷贝初始化
string nines = string(100, '9'); // 拷贝初始化
```

当使用直接初始化时，我们实际上是要求编译器使用普通的函数匹配（参见 6.4 节，第 209 页）来选择与我们提供的参数最匹配的构造函数。当我们使用拷贝初始化（copy initialization）时，我们要求编译器将右侧运算对象拷贝到正在创建的对象中，如果需要的话还要进行类型转换（参见 7.5.4 节，第 263 页）。

拷贝初始化通常使用拷贝构造函数来完成。但是，如我们将在 13.6.2 节（第 473 页）所见，如果一个类有一个移动构造函数，则拷贝初始化有时会使用移动构造函数而非拷贝构造函数来完成。但现在，我们只需了解拷贝初始化何时发生，以及拷贝初始化是依靠拷贝构造函数或移动构造函数来完成的就可以了。

拷贝初始化不仅在我们用=定义变量时会发生，在下列情况下也会发生

- 将一个对象作为实参传递给一个非引用类型的形参 只有非引用类型时才会发生拷贝，否则就是直接绑定了。
- 从一个返回类型为非引用类型的函数返回一个对象 直接绑定了。
- 用花括号列表初始化一个数组中的元素或一个聚合类中的成员（参见 7.5.5 节，第 266 页）

某些类类型还会对它们所分配的对象使用拷贝初始化。例如，当我们初始化标准库容器或是调用其 `insert` 或 `push` 成员（参见 9.3.1 节，第 306 页）时，容器会对其元素进行拷贝初始化。与之相对，用 `emplace` 成员创建的元素都进行直接初始化（参见 9.3.1 节，第 308 页）。

参数和返回值

在函数调用过程中，具有非引用类型的参数要进行拷贝初始化（参见 6.2.1 节，第 188 页）。类似的，当一个函数具有非引用的返回类型时，返回值会被用来初始化调用方的结果（参见 6.3.2 节，第 201 页）。

拷贝构造函数被用来初始化非引用类类型参数，这一特性解释了为什么拷贝构造函数自己的参数必须是引用类型。如果其参数不是引用类型，则调用永远也不会成功——为了调用拷贝构造函数，我们必须拷贝它的实参，但为了拷贝实参，我们又需要调用拷贝构造函数，如此无限循环。

拷贝初始化的限制

如前所述，如果我们使用的初始化值要求通过一个 `explicit` 的构造函数来进行类型转换（参见 7.5.4 节，第 265 页），那么使用拷贝初始化还是直接初始化就不是无关紧要的了：

```
vector<int> v1(10); // 正确：直接初始化
vector<int> v2 = 10; // 错误：接受大小参数的构造函数是 explicit 的
void f(vector<int>); // f 的参数进行拷贝初始化
f(10); // 错误：不能用一个 explicit 的构造函数拷贝一个实参
f(vector<int>(10)); // 正确：从一个 int 直接构造一个临时 vector
```

直接初始化 `v1` 是合法的，但看起来与之等价的拷贝初始化 `v2` 则是错误的，因为 `vector` 的接受单一大小参数的构造函数是 `explicit` 的。出于同样的原因，当传递一个实参或从函数返回一个值时，我们不能隐式使用一个 `explicit` 构造函数。如果我们希望使用一个 `explicit` 构造函数，就必须显式地使用，像此代码中最后一行那样。

499 编译器可以绕过拷贝构造函数

在拷贝初始化过程中，编译器可以（但不是必须）跳过拷贝/移动构造函数，直接创建对象。即，编译器被允许将下面的代码

```
string null_book = "9-999-99999-9"; // 拷贝初始化
```

改写为

```
string null_book("9-999-99999-9"); // 编译器略过了拷贝构造函数
```

但是，即使编译器略过了拷贝/移动构造函数，但在这个程序点上，拷贝/移动构造函数必须是存在且可访问的（例如，不能是 `private` 的）。

13.1.1 节练习

练习 13.1：拷贝构造函数是什么？什么时候使用它？



练习 13.2：解释为什么下面的声明是非法的：




```
Sales_data::Sales_data(Sales_data rhs);
```



练习 13.3: 当我们拷贝一个 `StrBlob` 时, 会发生什么? 拷贝一个 `StrBlobPtr` 呢?

练习 13.4: 假定 `Point` 是一个类类型, 它有一个 `public` 的拷贝构造函数, 指出下面程序片段中哪些地方使用了拷贝构造函数:

```
Point global;
Point foo_bar(Point arg)
{
    Point local = arg, *heap = new Point(global);
    *heap = local;
    Point pa[ 4 ] = { local, *heap };
    return *heap;
}
```

练习 13.5: 给定下面的类框架, 编写一个拷贝构造函数, 拷贝所有成员。你的构造函数应该动态分配一个新的 `string` (参见 12.1.2 节, 第 407 页), 并将对象拷贝到 `ps` 指向的位置, 而不是 `ps` 本身的位置。

```
class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
private:
    std::string *ps;
    int i;
};
```

13.1.2 拷贝赋值运算符



与类控制其对象如何初始化一样, 类也可以控制其对象如何赋值:

```
Sales_data trans, accum;
trans = accum; // 使用 Sales_data 的拷贝赋值运算符
```

与拷贝构造函数一样, 如果类未定义自己的拷贝赋值运算符, 编译器会为它合成一个。

重载赋值运算符

在介绍合成赋值运算符之前, 我们需要了解一点儿有关**重载运算符** (overloaded operator) 的知识, 详细内容将在第 14 章中进行介绍。

重载运算符本质上是函数, 其名字由 `operator` 关键字后接表示要定义的运算符的符号组成。因此, 赋值运算符就是一个名为 `operator=` 的函数。类似于任何其他函数, 运算符函数也有一个返回类型和一个参数列表。

重载运算符的参数表示运算符的运算对象。某些运算符, 包括赋值运算符, 必须定义为成员函数。如果一个运算符是一个成员函数, 其左侧运算对象就绑定到隐式的 `this` 参数 (参见 7.1.2 节, 第 231 页)。对于一个二元运算符, 例如赋值运算符, 其右侧运算对象作为显式参数传递。

拷贝赋值运算符接受一个与其所在类相同类型的参数:

```
class Foo {
public:
```

```

    Foo& operator=(const Foo&); // 赋值运算符
    // ...
};

```

为了与内置类型的赋值（参见 4.4 节，第 129 页）保持一致，赋值运算符通常返回一个指向其左侧运算对象的引用。另外值得注意的是，标准库通常要求保存在容器中的类型要具有赋值运算符，且其返回值是左侧运算对象的引用。



赋值运算符通常应该返回一个指向其左侧运算对象的引用。

合成拷贝赋值运算符

与处理拷贝构造函数一样，如果一个类未定义自己的拷贝赋值运算符，编译器会为它生成一个合成拷贝赋值运算符（synthesized copy-assignment operator）。类似拷贝构造函数，对于某些类，合成拷贝赋值运算符用来禁止该类型对象的赋值（参见 13.1.6 节，第 450 页）。如果拷贝赋值运算符并非出于此目的，它会将右侧运算对象的每个非 static 成员赋予左侧运算对象的对应成员，这一工作是通过成员类型的拷贝赋值运算符来完成的。对于数组类型的成员，逐个赋值数组元素。合成拷贝赋值运算符返回一个指向其左侧运算对象的引用。

501

作为一个例子，下面的代码等价于 Sales_data 的合成拷贝赋值运算符：

```

// 等价于合成拷贝赋值运算符
Sales_data&
Sales_data::operator=(const Sales_data &rhs)
{
    bookNo = rhs.bookNo;           // 调用 string::operator=
    units_sold = rhs.units_sold;    // 使用内置的 int 赋值
    revenue = rhs.revenue;          // 使用内置的 double 赋值
    return *this;                   // 返回一个此对象的引用
}

```

13.1.2 节练习

练习 13.6: 拷贝赋值运算符是什么？什么时候使用它？合成拷贝赋值运算符完成什么工作？什么时候会生成合成拷贝赋值运算符？

练习 13.7: 当我们将一个 StrBlob 赋值给另一个 StrBlob 时，会发生什么？赋值 StrBlobPtr 呢？

练习 13.8: 为 13.1.1 节（第 443 页）练习 13.5 中的 HasPtr 类编写赋值运算符。类似拷贝构造函数，你的赋值运算符应该将对象拷贝到 ps 指向的位置。



13.1.3 析构函数

析构函数执行与构造函数相反的操作：构造函数初始化对象的非 static 数据成员，还可能做一些其他工作；析构函数释放对象使用的资源，并销毁对象的非 static 数据成员。

析构函数是类的一个成员函数，名字由波浪号接类名构成。它没有返回值，也不接受参数：

```

class Foo {
public:
    ~Foo(); // 析构函数
}

```

```
//...
};
```

由于析构函数不接受参数，因此它不能被重载。对一个给定类，只会有唯一一个析构函数。

析构函数完成什么工作

如同构造函数有一个初始化部分和一个函数体（参见 7.5.1 节，第 257 页），析构函数也有一个函数体和一个析构部分。在一个构造函数中，成员的初始化是在函数体执行之前完成的，且按照它们在类中出现的顺序进行初始化。在一个析构函数中，首先执行函数体，502然后销毁成员。成员按初始化顺序的逆序销毁。

在对象最后一次使用之后，析构函数的函数体可执行类设计者希望执行的任何收尾工作。通常，析构函数释放对象在生存期分配的所有资源。

在一个析构函数中，不存在类似构造函数中初始化列表的东西来控制成员如何销毁，析构部分是隐式的。成员销毁时发生什么完全依赖于成员的类型。销毁类类型的成员需要执行成员自己的析构函数。内置类型没有析构函数，因此销毁内置类型成员什么也不需要

Note

隐式销毁一个内置指针类型的成员不会 delete 它所指向的对象。

与普通指针不同，智能指针（参见 12.1.1 节，第 402 页）是类类型，所以具有析构函数。因此，与普通指针不同，智能指针成员在析构阶段会被自动销毁。

什么时候会调用析构函数

无论何时一个对象被销毁，就会自动调用其析构函数：

- 变量在离开其作用域时被销毁。
- 当一个对象被销毁时，其成员被销毁。
- 容器（无论是标准库容器还是数组）被销毁时，其元素被销毁。
- 对于动态分配的对象，当对指向它的指针应用 delete 运算符时被销毁（参见 12.1.2 节，第 409 页）。
- 对于临时对象，当创建它的完整表达式结束时被销毁。

由于析构函数自动运行，我们的程序可以按需要分配资源，而（通常）无须担心何时释放这些资源。

例如，下面代码片段定义了四个 Sales_data 对象：

```
{ // 新作用域
    // p 和 p2 指向动态分配的对象
    Sales_data *p = new Sales_data;          // p 是一个内置指针
    auto p2 = make_shared<Sales_data>();    // p2 是一个 shared_ptr
    Sales_data item(*p);                    // 拷贝构造函数将 *p 拷贝到 item 中
    vector<Sales_data> vec;                 // 局部对象
    vec.push_back(*p2);                     // 拷贝 p2 指向的对象
    delete p;                               // 对 p 指向的对象执行析构函数
} // 退出局部作用域；对 item、p2 和 vec 调用析构函数
// 销毁 p2 会递减其引用计数；如果引用计数变为 0，对象被释放
// 销毁 vec 会销毁它的元素
```

503 每个 `Sales_data` 对象都包含一个 `string` 成员，它分配动态内存来保存 `bookNo` 成员中的字符。但是，我们的代码唯一需要直接管理的内存就是我们直接分配的 `Sales_data` 对象。我们的代码只需直接释放绑定到 `p` 的动态分配对象。

其他 `Sales_data` 对象会在离开作用域时被自动销毁。当程序块结束时，`vec`、`p2` 和 `item` 都离开了作用域，意味着在这些对象上分别会执行 `vector`、`shared_ptr` 和 `Sales_data` 的析构函数。`vector` 的析构函数会销毁我们添加到 `vec` 的元素。`shared_ptr` 的析构函数会递减 `p2` 指向的对象的引用计数。在本例中，引用计数会变为 0，因此 `shared_ptr` 的析构函数会 `delete` `p2` 分配的 `Sales_data` 对象。

在所有情况下，`Sales_data` 的析构函数都会隐式地销毁 `bookNo` 成员。销毁 `bookNo` 会调用 `string` 的析构函数，它会释放用来保存 ISBN 的内存。



当指向一个对象的引用或指针离开作用域时，析构函数不会执行。

合成析构函数

当一个类未定义自己的析构函数时，编译器会为它定义一个合成析构函数 (synthesized destructor)。类似拷贝构造函数和拷贝赋值运算符，对于某些类，合成析构函数被用来阻止该类型的对象被销毁（参见 13.1.6 节，第 450 页）。如果不是这种情况，合成析构函数的函数体就为空。

例如，下面的代码片段等价于 `Sales_data` 的合成析构函数：

```
class Sales_data {
public:
    // 成员会被自动销毁，除此之外不需要做其他事情
    ~Sales_data() { }
    // 其他成员的定义，如前
};
```

在（空）析构函数体执行完毕后，成员会被自动销毁。特别的，`string` 的析构函数会被调用，它将释放 `bookNo` 成员所用的内存。

认识到析构函数体自身并不直接销毁成员是非常重要的。成员是在析构函数体之后隐含的析构阶段中被销毁的。在整个对象销毁过程中，析构函数体是作为成员销毁步骤之外的另一部分而进行的。

13.1.3 节练习

练习 13.9: 析构函数是什么？合成析构函数完成什么工作？什么时候会生成合成析构函数？

练习 13.10: 当一个 `StrBlob` 对象销毁时会发生什么？一个 `StrBlobPtr` 对象销毁时呢？

练习 13.11: 为前面练习中的 `HasPtr` 类添加一个析构函数。

练习 13.12: 在下面的代码片段中会发生几次析构函数调用？

```
bool fcn(const Sales_data *trans, Sales_data accum)
{
    Sales_data item1(*trans), item2(accum);
```

```

        return item1.isbn() != item2.isbn();
    }

```

练习 13.13: 理解拷贝控制成员和构造函数的一个好方法是定义一个简单的类，为该定义这些成员，每个成员都打印出自己的名字：

```

struct X {
    X() {std::cout << "X()" << std::endl;}
    X(const X&) {std::cout << "X(const X&)" << std::endl;}
};

```

给 `x` 添加拷贝赋值运算符和析构函数，并编写一个程序以不同方式使用 `x` 的对象：将它们作为非引用和引用参数传递；动态分配它们；将它们存放于容器中；诸如此类。观察程序的输出，直到你确认理解了什么时候会使用拷贝控制成员，以及为什么会使用它们。当你观察程序输出时，记住编译器可以略过对拷贝构造函数的调用。

13.1.4 三/五法则



如前所述，有三个基本操作可以控制类的拷贝操作：拷贝构造函数、拷贝赋值运算符和析构函数。而且，在新标准下，一个类还可以定义一个移动构造函数和一个移动赋值运算符，我们将在 13.6 节（第 470 页）中介绍这些内容。

C++ 语言并不要求我们定义所有这些操作：可以只定义其中一个或两个，而不必定义所有。但是，这些操作通常应该被看作一个整体。通常，只需要其中一个操作，而不需要定义所有操作的情况是很少见的。

504

需要析构函数的类也需要拷贝和赋值操作

当我们决定一个类是否需要定义它自己版本的拷贝控制成员时，一个基本原则是首先确定这个类是否需要一个析构函数。通常，对析构函数的需求要比对拷贝构造函数或赋值运算符的需求更为明显。如果这个类需要一个析构函数，我们几乎可以肯定它也需要一个拷贝构造函数和一个拷贝赋值运算符。

我们在练习中用过的 `HasPtr` 类是一个好例子（参见 13.1.1 节，第 443 页）。这个类在构造函数中分配动态内存。合成析构函数不会 `delete` 一个指针数据成员。因此，此类需要定义一个析构函数来释放构造函数分配的内存。

应该怎么做可能还有点儿不清晰，但基本原则告诉我们，`HasPtr` 也需要一个拷贝构造函数和一个拷贝赋值运算符。

如果我们为 `HasPtr` 定义一个析构函数，但使用合成版本的拷贝构造函数和拷贝赋值运算符，考虑会发生什么：

505

```

class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
    ~HasPtr() { delete ps; }
    // 错误: HasPtr 需要一个拷贝构造函数和一个拷贝赋值运算符
    // 其他成员的定义，如前
};

```

在这个版本的类定义中，构造函数中分配的内存将在 `HasPtr` 对象销毁时被释放。但不幸的是，我们引入了一个严重的错误！这个版本的类使用了合成的拷贝构造函数和拷贝

赋值运算符。这些函数简单拷贝指针成员,这意味着多个 HasPtr 对象可能指向相同的内存:

```
HasPtr f(HasPtr hp)      // HasPtr 是传值参数, 所以将被拷贝
{
    HasPtr ret = hp;      // 拷贝给定的 HasPtr
    // 处理 ret
    return ret;           // ret 和 hp 被销毁
}
```

当 f 返回时, hp 和 ret 都被销毁, 在两个对象上都会调用 HasPtr 的析构函数。此析构函数会 delete ret 和 hp 中的指针成员。但这两个对象包含相同的指针值。此代码会导致此指针被 delete 两次, 这显然是一个错误 (参见 12.1.2 节, 第 411 页)。将要发生什么是未定义的。

此外, f 的调用者还会使用传递给 f 的对象:

```
HasPtr p("some values");
f(p);                // 当 f 结束时, p.ps 指向的内存被释放
HasPtr q(p);         // 现在 p 和 q 都指向无效内存!
```

p (以及 q) 指向的内存不再有效, 在 hp (或 ret!) 销毁时它就被归还给系统了。



如果一个类需要自定义析构函数, 几乎可以肯定它也需要自定义拷贝赋值运算符和拷贝构造函数。

需要拷贝操作的类也需要赋值操作, 反之亦然

虽然很多类需要定义所有 (或是不需要定义任何) 拷贝控制成员, 但某些类所要完成的工作, 只需要拷贝或赋值操作, 不需要析构函数。

作为一个例子, 考虑一个类为每个对象分配一个独有的、唯一的序号。这个类需要一个拷贝构造函数为每个新创建的对象生成一个新的、独一无二的序号。除此之外, 这个拷贝构造函数从给定对象拷贝所有其他数据成员。这个类还需要自定义拷贝赋值运算符来避免将序号赋予目的对象。但是, 这个类不需要自定义析构函数。

这个例子引出了第二个基本原则: 如果一个类需要一个拷贝构造函数, 几乎可以肯定它也需要一个拷贝赋值运算符。反之亦然——如果一个类需要一个拷贝赋值运算符, 几乎可以肯定它也需要一个拷贝构造函数。然而, 无论是需要拷贝构造函数还是需要拷贝赋值运算符都不必然意味着也需要析构函数。

13.1.4 节练习

练习 13.14: 假定 numbered 是一个类, 它有一个默认构造函数, 能为每个对象生成一个唯一的序号, 保存在名为 mysn 的数据成员中。假定 numbered 使用合成的拷贝控制成员, 并给定如下函数:

```
void f (numbered s) { cout << s.mysn << endl; }
```

则下面代码输出什么内容?

```
numbered a, b = a, c = b;
f(a); f(b); f(c);
```

练习 13.15: 假定 numbered 定义了一个拷贝构造函数, 能生成一个新的序号。这会改变上一题中调用的输出结果吗? 如果会改变, 为什么? 新的输出结果是什么?

练习 13.16: 如果 `f` 中的参数是 `const numbered&`, 将会怎样? 这会改变输出结果吗? 如果会改变, 为什么? 新的输出结果是什么?

练习 13.17: 分别编写前三题中所描述的 `numbered` 和 `f`, 验证你是否正确预测了输出结果。

13.1.5 使用=default

我们可以通过将拷贝控制成员定义为 `=default` 来显式地要求编译器生成合成的版本 (参见 7.1.4 节, 第 237 页):

C++
11

```
class Sales_data {
public:
    // 拷贝控制成员; 使用 default
    Sales_data() = default;
    Sales_data(const Sales_data&) = default;
    Sales_data& operator=(const Sales_data &);
    ~Sales_data() = default;
    // 其他成员的定义, 如前
};
Sales_data& Sales_data::operator=(const Sales_data&) = default;
```

当我们在类内用 `=default` 修饰成员的声明时, 合成的函数将隐式地声明为内联的 (就像任何其他类内声明的成员函数一样)。如果我们不希望合成的成员是内联函数, 应该只对成员的类外定义使用 `=default`, 就像对拷贝赋值运算符所做的那样。

507



我们只能对具有合成版本的成员函数使用 `=default` (即, 默认构造函数或拷贝控制成员)。

13.1.6 阻止拷贝



大多数类应该定义默认构造函数、拷贝构造函数和拷贝赋值运算符, 无论是隐式地还是显式地。

虽然大多数类应该定义 (而且通常也的确定义了) 拷贝构造函数和拷贝赋值运算符, 但对某些类来说, 这些操作没有合理的意义。在此情况下, 定义类时必须采用某种机制阻止拷贝或赋值。例如, `iostream` 类阻止了拷贝, 以避免多个对象写入或读取相同的 IO 缓冲。为了阻止拷贝, 看起来可能应该不定义拷贝控制成员。但是, 这种策略是无效的: 如果我们的类未定义这些操作, 编译器为它生成合成的版本。

定义删除的函数

在新标准下, 我们可以通过将拷贝构造函数和拷贝赋值运算符定义为删除的函数 (deleted function) 来阻止拷贝。删除的函数是这样一种函数: 我们虽然声明了它们, 但不能以任何方式使用它们。在函数的参数列表后面加上 `=delete` 来指出我们希望将它定义为删除的:

C++
11

```
struct NoCopy {
    NoCopy() = default;           // 使用合成的默认构造函数
    NoCopy(const NoCopy&) = delete; // 阻止拷贝
    NoCopy &operator=(const NoCopy&) = delete; // 阻止赋值
```

```

    ~NoCopy() = default;    // 使用合成的析构函数
    // 其他成员
};

```

`=delete` 通知编译器（以及我们代码的读者），我们不希望定义这些成员。

与 `=default` 不同，`=delete` 必须出现在函数第一次声明的时候，这个差异与这些声明的含义在逻辑上是吻合的。一个默认的成员只影响为这个成员而生成的代码，因此 `=default` 直到编译器生成代码时才需要。而另一方面，编译器需要知道一个函数是删除的，以便禁止试图使用它的操作。

与 `=default` 的另一个不同之处是，我们可以对任何函数指定 `=delete`（我们只能对编译器可以合成的默认构造函数或拷贝控制成员使用 `=default`）。虽然删除函数的主要用途是禁止拷贝控制成员，但当我们希望引导函数匹配过程时，删除函数有时也是有用的。

析构函数不能是删除的成员

值得注意的是，我们不能删除析构函数。如果析构函数被删除，就无法销毁此类型的对象了。对于一个删除了析构函数的类型，编译器将不允许定义该类型的变量或创建该类的临时对象。而且，如果一个类有某个成员的类型删除了析构函数，我们也不能定义该类的变量或临时对象。因为如果一个成员的析构函数是删除的，则该成员无法被销毁。而如果一个成员无法被销毁，则对象整体也就无法被销毁了。

对于删除了析构函数的类型，虽然我们不能定义这种类型的变量或成员，但可以动态分配这种类型的对象。但是，不能释放这些对象：

```

struct NoDtor {
    NoDtor() = default;    // 使用合成默认构造函数
    ~NoDtor() = delete;    // 我们不能销毁 NoDtor 类型的对象
};
NoDtor nd; // 错误：NoDtor 的析构函数是删除的
NoDtor *p = new NoDtor(); // 正确：但我们不能 delete p
delete p; // 错误：NoDtor 的析构函数是删除的

```



对于析构函数已删除的类型，不能定义该类型的变量或释放指向该类型动态分配对象的指针。

合成的拷贝控制成员可能是删除的

如前所述，如果我们未定义拷贝控制成员，编译器会为我们定义合成的版本。类似的，如果一个类未定义构造函数，编译器会为其合成一个默认构造函数（参见 7.1.4 节，第 235 页）。对某些类来说，编译器将这些合成的成员定义为删除的函数：

- 如果类的某个成员的析构函数是删除的或不可访问的（例如，是 `private` 的），则类的合成析构函数被定义为删除的。
- 如果类的某个成员的拷贝构造函数是删除的或不可访问的，则类的合成拷贝构造函数被定义为删除的。如果类的某个成员的析构函数是删除的或不可访问的，则类合成的拷贝构造函数也被定义为删除的。
- 如果类的某个成员的拷贝赋值运算符是删除的或不可访问的，或是类有一个 `const` 的或引用成员，则类的合成拷贝赋值运算符被定义为删除的。
- 如果类的某个成员的析构函数是删除的或不可访问的，或是类有一个引用成员，它没有类内初始化器（参见 2.6.1 节，第 65 页），或是类有一个 `const` 成员，它没有

类内初始化器且其类型未显式定义默认构造函数，则该类的默认构造函数被定义为删除的。

本质上，这些规则的含义是：如果一个类有数据成员不能默认构造、拷贝、复制或销毁，◀ 509则对应的成员函数将被定义为删除的。

一个成员有删除的或不可访问的析构函数会导致合成的默认和拷贝构造函数被定义为删除的，这看起来可能有些奇怪。其原因是，如果没有这条规则，我们可能会创建出无法销毁的对象。

对于具有引用成员或无法默认构造的 `const` 成员的类，编译器不会为其合成默认构造函数，这应该不奇怪。同样不出人意料规则是：如果一个类有 `const` 成员，则它不能使用合成的拷贝赋值运算符。毕竟，此运算符试图赋值所有成员，而将一个新值赋予一个 `const` 对象是不可能的。

虽然我们可以将一个新值赋予一个引用成员，但这样做改变的是引用指向的对象的值，而不是引用本身。如果为这样的类合成拷贝赋值运算符，则赋值后，左侧运算对象仍然指向与赋值前一样的对象，而不会与右侧运算对象指向相同的对象。由于这种行为看起来并不是我们所期望的，因此对于有引用成员的类，合成拷贝赋值运算符被定义为删除的。

我们将在 13.6.2 节（第 476 页）、15.7.2 节（第 553 页）及 19.6 节（第 751 页）中介绍导致类的拷贝控制成员被定义为删除函数的其他原因。



本质上，当不可能拷贝、赋值或销毁类的成员时，类的合成拷贝控制成员就被定义为删除的。

private 拷贝控制

在新标准发布之前，类是通过将其拷贝构造函数和拷贝赋值运算符声明为 `private` 的来阻止拷贝：

```
class PrivateCopy {  
    // 无访问说明符；接下来的成员默认为 private 的；参见 7.2 节（第 240 页）  
    // 拷贝控制成员是 private 的，因此普通用户代码无法访问  
    PrivateCopy(const PrivateCopy&);  
    PrivateCopy &operator=(const PrivateCopy&);  
    // 其他成员  
public:  
    PrivateCopy() = default; // 使用合成的默认构造函数  
    ~PrivateCopy(); // 用户可以定义此类型的对象，但无法拷贝它们  
};
```

由于析构函数是 `public` 的，用户可以定义 `PrivateCopy` 类型的对象。但是，由于拷贝构造函数和拷贝赋值运算符是 `private` 的，用户代码将不能拷贝这个类型的对象。但是，友元和成员函数仍旧可以拷贝对象。为了阻止友元和成员函数进行拷贝，我们将这些拷贝控制成员声明为 `private` 的，但并不定义它们。

声明但不定义一个成员函数是合法的（参见 6.1.2 节，第 186 页），对此只有一个例外，我们将在 15.2.1 节（第 528 页）中介绍。试图访问一个未定义的成员将导致一个链接时错误。通过声明（但不定义）`private` 的拷贝构造函数，我们可以预先阻止任何拷贝该类型对象的企图：试图拷贝对象的用户代码将在编译阶段被标记为错误；成员函数或友函数中的拷贝操作将会导致链接时错误。◀ 510



希望阻止拷贝的类应该使用 `=delete` 来定义它们自己的拷贝构造函数和拷贝赋值运算符，而不应该将它们声明为 `private` 的。

13.1.6 节练习

练习 13.18: 定义一个 `Employee` 类，它包含雇员的姓名和唯一的雇员证号。为这个类定义默认构造函数，以及接受一个表示雇员姓名的 `string` 的构造函数。每个构造函数应该通过递增一个 `static` 数据成员来生成一个唯一的证号。

练习 13.19: 你的 `Employee` 类需要定义它自己的拷贝控制成员吗？如果需要，为什么？如果不需要，为什么？实现你认为 `Employee` 需要的拷贝控制成员。

练习 13.20: 解释当我们拷贝、赋值或销毁 `TextQuery` 和 `QueryResult` 类（参见 12.3 节，第 430 页）对象时会发生什么。

练习 13.21: 你认为 `TextQuery` 和 `QueryResult` 类需要定义它们自己版本的拷贝控制成员吗？如果需要，为什么？如果不需要，为什么？实现你认为这两个类需要的拷贝控制操作。



13.2 拷贝控制和资源管理

通常，管理类外资源的类必须定义拷贝控制成员。如我们在 13.1.4 节（第 447 页）中所见，这类类需要通过析构函数来释放对象所分配的资源。一旦一个类需要析构函数，那么它几乎肯定也需要一个拷贝构造函数和一个拷贝赋值运算符。

为了定义这些成员，我们首先必须确定此类型对象的拷贝语义。一般来说，有两种选择：可以定义拷贝操作，使类的行为看起来像一个值或者像一个指针。

类的行为像一个值，意味着它应该也有自己的状态。当我们拷贝一个像值的对象时，副本和原对象是完全独立的。改变副本不会对原对象有任何影响，反之亦然。

行为像指针的类则共享状态。当我们拷贝一个这种类的对象时，副本和原对象使用相同的底层数据。改变副本也会改变原对象，反之亦然。

在我们使用过的标准库类中，标准库容器和 `string` 类的行为像一个值。而不出意外的，`shared_ptr` 类提供类似指针的行为，就像我们的 `StrBlob` 类（参见 12.1.1 节，第 405 页）一样，`IO` 类型和 `unique_ptr` 不允许拷贝或赋值，因此它们的行为既不像值也不像指针。

为了说明这两种方式，我们会为练习中的 `HasPtr` 类定义拷贝控制成员。首先，我们将令类的行为像一个值；然后重新实现类，使它的行为像一个指针。

我们的 `HasPtr` 类有两个成员，一个 `int` 和一个 `string` 指针。通常，类直接拷贝内置类型（不包括指针）成员；这些成员本身就是值，因此通常应该让它们的行为像值一样。我们如何拷贝指针成员决定了像 `HasPtr` 这样的类是具有类值行为还是类指针行为。

13.2 节练习

练习 13.22: 假定我们希望 `HasPtr` 的行为像一个值。即，对于对象所指向的 `string`